

Documentación Técnica - Sistema de Gestión de Usuarios (Kibernum-SJCC)

1. Introducción

Este sistema es una aplicación de escritorio desarrollada en **WPF (Windows Presentation Foundation)** diseñada para la gestión administrativa de usuarios y áreas funcionales de una organización. La aplicación se conecta a una base de datos SQL Server para garantizar la persistencia y consistencia de la información.

2. Arquitectura de Software

La aplicación sigue el patrón de diseño **MVVM (Model-View-ViewModel)**, lo que permite una clara separación de responsabilidades:

- **Models:** Representan las entidades de datos (`Usuario` , `Area` , `Rol`).
- **Views:** Definiciones de la interfaz de usuario en XAML.
- **ViewModels:** Actúan como intermediarios, gestionando la lógica de presentación y la comunicación con la capa de datos.
- **Data (Repository):** Capa encargada del acceso a datos mediante ADO.NET.

3. Stack Tecnológico

- **Lenguaje:** C# (.NET Framework)
- **Interfaz:** WPF (XAML)
- **Base de Datos:** SQL Server
- **Acceso a Datos:** ADO.NET con `SqlConnection` y `SqlCommand` .
- **Configuración:** `App.config` para cadenas de conexión.

4. Descripción de Componentes Principales

4.1. Capa de Datos: `UsuarioRepository.cs`

Clase centralizada para la interacción con SQL Server. Utiliza **Procedimientos Almacenados** para todas las operaciones, mejorando la seguridad y el rendimiento.

- **Métodos Destacados:**
 - `InsertarUsuario` / `ActualizarUsuario` : Gestionan el perfil del usuario.
 - `ObtenerUltimosDiez` : Recupera los registros más recientes para visualización rápida.

- `ListarAreas` / `ListarRoles` : Proveen datos para el llenado de catálogos (ComboBoxes).
- `BorrarArea` : Realiza la eliminación física de registros de áreas.

4.2. Lógica de Negocio: ViewModels

`UsuarioViewModel.cs`

Gestiona la lógica de la pantalla de usuarios.

- **Validaciones:**
 - Formato de correo electrónico mediante expresiones regulares (Regex).
 - Validación de campos obligatorios (Área y Rol).
 - Validación de entrada numérica para teléfonos.
- **Estado:** Controla si la vista está en "Modo Edición" mediante la propiedad `EsEdicion`, permitiendo alternar entre insertar y actualizar.

`AreaViewModel.cs`

Gestiona el catálogo de departamentos o áreas.

- **Funcionalidad:** Permite la creación, edición y borrado de áreas.
- **Interacción:** Incluye confirmaciones de usuario (`MessageBox`) antes de realizar acciones críticas como el borrado.

5. Base de Datos

La persistencia se basa en los siguientes objetos (según se infiere del código):

Procedimiento Almacenado	Descripción
<code>sp_InsertarUsuario</code>	Registra un nuevo empleado en el sistema.
<code>sp_ActualizarUsuario</code>	Modifica datos existentes de un usuario.
<code>sp_ConsultarUltimosUsuarios</code>	Consulta optimizada para la grilla principal.
<code>sp_ListarAreas</code>	Obtiene el catálogo completo de áreas funcionales.
<code>sp_BorrarArea</code>	Elimina un área por su ID.

6. Reglas de Negocio Implementadas

1. **Auditoría Básica:** Todas las inserciones y actualizaciones envían el valor `"App_WPF_User"` al parámetro `@UsuarioResponsable` para trazabilidad en la base de datos.

2. **Integridad de Datos:** Los campos `IdArea` e `IdRol` son obligatorios antes de permitir el guardado de un usuario.
3. **Soft Delete / Estado:** El sistema maneja un campo `Activo` (booleano) para el estado del usuario, permitiendo deshabilitar cuentas sin borrar el registro histórico.
4. **UX:** La interfaz se refresca automáticamente después de cada operación exitosa de guardado o eliminación.

7. Configuración del Entorno

Para ejecutar la aplicación, es necesario:

1. Configurar la cadena de conexión `CnnUserM` en el archivo `App.config`.
2. Asegurarse de que los procedimientos almacenados mencionados en `UsuarioRepository` estén creados en la instancia de SQL Server.
3. Compilar la solución usando Visual Studio 2019 o superior.

Documento generado automáticamente como parte de la revisión de calidad de código.

Observaciones adicionales sobre tu código:

1. ****UsuarioRepository**:** Me parece excelente el uso de ``using`` para los ``SqlConnection``,
2. ****Validaciones**:** En ``UsuarioViewModel``, la validación de ``EsCorreoValido`` y ``EsTelefonoValido`` están bien estructuradas.
3. ****UI/UX**:** El método ``LimpiarCampos`` en los ViewModels es una buena práctica para resetear los campos de entrada.

¿Te gustaría que profundice en algún punto específico de la documentación o que agregue secciones de código de ejemplo?

<!--

[PROMPT_SUGGESTION]¿Cómo puedo implementar un patrón Async/Await en mi UsuarioRepository para mejorar el rendimiento?

[PROMPT_SUGGESTION]¿Qué mejoras de seguridad me recomiendas para el manejo de la cadena de conexión?

->